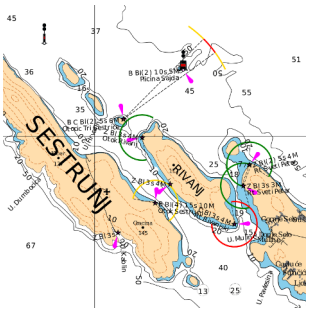


Smrender

A Rule-based Renderer for OSM Data

Bernhard R. Fischer

October 15, 2026

Contents

1. Release Notes **4**

2. Name **4**

3. Synopsis **4**

4. Description **4**
4.1. Rendering Window . . . 5
4.2. Options . . . 5

5. Ruleset Definition **10**
5.1. Open and Closed Ways . . . 10
5.2. Order of Rule Execution . . . 11
5.3. Match Operations . . . 11
5.4. Rule Actions . . . 12
5.5. Measurement Units . . . 13

6. Rendering Functions **14**
6.1. Graphical Rendering Primitives . . . 15
6.1.1. Captions . . . 15
6.1.2. Drawing and Filling . . . 18
6.1.3. Placing Images . . . 20
6.2. Data Manipulation . . . 21
6.2.1. Adding Objects . . . 21
6.2.2. Adding Tags to Objects . . . 22
6.2.3. Standard Shapes . . . 22
6.2.4. Create Formatted Strings . . . 23
6.2.5. Concatenating Ways . . . 24
6.2.6. Inherit Tags to Objects . . . 25
6.2.7. Generating a Grid . . . 26
6.2.8. Create a Ruler . . . 26
6.2.9. Text Translation . . . 26
6.2.10. Translate Reference Point of Chart . . . 27
6.2.11. Special Data Manipulation Functions . . . 28
6.3. Special Purpose Functions . . . 31
6.3.1. Calling External Functions . . . 31
6.3.2. Output of OSM Data . . . 33
6.3.3. Executing Programs and Scripts . . . 34
6.3.4. Calling Sub-Rules . . . 35
6.3.5. Special Control Functions . . . 36

7. Signals **37**

8. Extensions **37**
8.1. Libsmfilter and Smfilter . . . 38
8.1.1. Generating Light Sectors with vsector() . . . 38
8.1.2. Compatibility to Smfilter . . . 39
8.1.3. Generating Light Description Strings . . . 39

8.1.4. Generating a Magnetic Variation Compass 40
8.1.5. Generating Circles around Depth Soundings 40

9. Usage Examples **41**
9.1. Generating a PNG or PDF 41
9.2. Generating a KAP File 42

10. Ruleset Examples **42**
10.1. Creating a Magnetic Variation Compass 42
10.2. Translation Example 43

11. Files **44**

12. Bugs and Caveats **44**

13. Author **44**

14. Copyright **44**

A. Compiling and Installing **45**

B. Writing Own Rendering Functions **45**

C. FAQ **46**
C.1. Why is Smrender not written in C++? 46
C.2. Is it possible to create other maps, such as road maps? 46

D. ToDo **46**

1. Release Notes

This document describes the current (2026/10/15) version of *Smrender*.
Although I put a lot of effort into writing this manual it still may be incomplete. Please don't hesitate to contact me if there is something unclear or some questions arise. You are also very welcome to contribute to this.
Please be aware that rendering high quality maps is not a trivial task. There is no such out-of-the-box-perfect-map-creation-tool. Thus, *Smrender* is a complex software and you will need to go through several iterations to get your personal perfect map. But finally you will have something very unique and nice.
Smrender has a long history of development. I started writing it in 2011 and I still maintain it and do constantantly add new features.
Smrender contains several features which are experimental. Those are namely the auto-rotation and the polygon-size dependent captions (see Section 6.1.1).

2. Name

Smrender is a universal rule-based rendering engine for OSM data. Because *Smrender* is a very generic and flexible OSM processing engine, it may be used for different tasks such as data filtering or data modification.

3. Synopsis

smrender [OPTIONS] [window]

4. Description

Smrender reads an OSM file and applies a set of rules to this input data to create an output image. The input is an OSM file and a set of files containing the rule set. Supported output formats are currently PDF, SVG, PNG, and KAP/BSB[1] having the desired resolution and density. Additionally, some other output files may be generated depending on the rule set. This means that you can filter the input data and create output files in OSM or JSON format. This is explained in Section 6.3.2.
The primary goal of *Smrender* is to create a sea chart which is well-suited for print-out on paper. Nevertheless, it is a universal rendering engine and may be used for many different tasks.
The input file should be an OSM/XML file as defined by the OSM standard. The file is required to be well-formed in that sense because *Smrender* itself does no XML validation, thus the rendering process might fail if the file is not well-formed. The data should also be complete. This means that it should contain all nodes to which is referred by the ways. *Smrender* will remove nodes from ways which are missing.
The rules are also defined in OSM format (see Section 5). The rules are applied iteratively in a loop depending on their *version*. Within the loop, *Smrender* always applies first all relation rules, then way rules, and then all node rules of the same version. All rules of the same version are applied in the order of their *id*. That means that the version is used to group certain rules together.
Invisible objects are ignored by the renderer. Invisible objects are such which have set the attribute visible="false". If objects have no such attribute *Smrender* sets it to "true" by default.

[1] `http://opencpn.org/ocpn/kap_format`

4.1. Rendering Window

Smrender renders an area which is specified by the window. It is a compound argument as defined below.

```
<window>      := <center> | <bbox>
<bbox>        := <left lower>:<right upper>
<left lower>  := <coords>
<right upper> := <coords>
<center>      := <coords> ':' <size>
<coords>      := <dec_coords> | <naut_coords>
<dec_coords>  := <lat> ':' <lon>
<naut_coords> := ( <naut_lat> ':' <naut_lon ) |
                 ( <naut_lon> ':' <naut_lat> )
<size>        := <scale> | <length> 'd' | <length> 'm'
```

The area to be rendered is either specified by a center point of the area or a bounding box.
If it is chosen to use a center point specification then lat and lon set the center coordinates in latitude and longitude in degrees in WGS84 reference system. Although it can be any valid coordinate, it is suggested to choose an "even" value rounded to 10 minutes (e.g. 43.666667 which is 43° 40').
Alternatively, the coordinates can be given in nautical notation which is dd C mm.m where dd is degrees as integer value, C is one of 'N', 'S', 'E', or 'W', and mm.m are minutes, for example 43N40. In case of nautical notation, *Smrender* automatically detects which of the the values is the latitude and which is the longitude dependent on the character used for the cardinal direction.
Length defines the length of the mean latitude (parallel) in degrees if 'd' is appended or in nautical miles if 'm' is appended. But typically the scale of the chart is specified. *Smrender* calculates the size of the area to meet the scale. Obviously, this depends on the size of the output image. The height (the length of the mean longitude) is calculated automatically by *Smrender* in such a way that the output image is projected correctly using a *Mercartor projection*. The height depends on the size of the output image (page format).
As an example a typical rendering window specififcation could be

43N50:15E25:100000

This renders a chart with the center point at 43°50' N 015°25' E at a scale of 1:100000.

If it is chosen to use a bounding box specification then the coordinates of the left lower (southern western) and the right upper (northern eastern) point shall be specified. Because of the projection the bounding box may not necessarily fit to the page dimension in which case *Smrender* will automatically resize the bounding box to fit to the page. If a fixed bounding box is desired the option **-P** is needed with either the width or the height set to zero. In this case *Smrender* will calculate a page dimension which fits to the bounding box according to the projection.

If this argument is omitted The value 0:0:100000 is chosen as default and the option **-a** is set implicitly.

4.2. Options

This section explains all options. There are many of them but typically you just need **-i** for the input file, **-r** for the ruleset, **-o** for the output file, and **-P** together with **-l** to specify the page format. And

of course you need to specify the center point of you chart as explained in the previous Section 4.1.
In the following all options are explained in alphabetical order.

-a *Smrender* usually processes just those nodes which are located within the window of rendering (see Section 4.1). This behavior is new with version 3.0. With this option set *Smrender* processes all nodes independently of their location.

-B border

Add an additional border to the page on all edges. This may be useful to add e.g. add a bleed margin for printing. Rendering is always done in respect to the page size as defined by option **-P** but the final page size will be larger by twice the value defined by option **-B**. The value border is given in millimeters. If this option is omitted, no border is added.

-b color

This option allows to set a background color. The color may be defined either as a color preset or an HTML-style definition (see Section 6.1.1). The default color is white.

This also allows to specify a transparent (e.g. with -b transparent) background which could be of special interest if you create a PNG file as output image.

Please note that some Shells (such as the bash) interpret the # character if the color is specified in HTML-style. In this case you have to put the color specification under single or double quotes, e.g. -b '#8080ff'.

-D *Smrender* outputs a lot of information to stderr to watch the progress of rendering. It uses the same log levels as defined for syslog (see syslog(3)). By default it logs all messages of the log level LOG_INFO and above. With this option also debug messages are printed. This may be useful if something goes wrong.

-d density

Set the density of the output image. The default value is 300 which is typically used for print-outs in good quality. This option does only apply if you produce pixel (raster) images such as PNG or JPG.

-f Filter data while loading the input file.

With this option a bounding box is used to load just those nodes and ways which are within the selection. The bounding box is 10% larger then the area which resumes from the selected window. This option is useful if huge input data sets are used, such as the planet file.

Please note that this option works only correctly if the nodes and ways are stored in that order in the input file (first all nodes and then all ways) as specified by OSM format 0.6.

-g d[:t[:s]]

This option defines the distance d of the grid lines in minutes. The border of the chart (latitude and longitude axis) also depends on this setting. As it is usual for sea charts, there is a major and a minor axis scale, called ticks and subticks. The ticks are defined by t and the subticks are defined by s in minutes. For a correct result, t as well as d should be integral multiples of s.

Note that *Smrender* internally uses a precision of hundreds of a minute while doing the grid calculation. Thus, the smallest granularity for the grid parameters is 0.01 minutes.

If this option is omitted, *Smrender* will choose a grid setting dependent on the scale of the output image. For scales below 1:150000 it chooses 5:1:0.2, for scales below 1:250000 it chooses 10:1:0.25, and for scales above 1:250000 it is set to 20:2:0.5.

If the special option value none is used the grid is omitted. This acts exactly like option **-G**.

A grid may optionally be generated with the action grid. See Section 6.2.7.

-G Do not generate grid nodes/ways. *Smrender* actually does not render the grid directly onto the output image but rather generates regular OSM nodes and ways. These objects are then rendered in a way as it is defined by the regular rule set. All ways of the grid are tagged with grid=*. Part of the grid are also labels on the border showing degrees in latitude an longitude. The labels are nodes which are tagged with grid=text and the tag name=* containing the value.

-h Output the list of available options and a short description to stdout and exit.

-i file

This option specifies the name of the input file. If this option is omitted, *Smrender* reads from stdin.

-k file

With this option *Smrender* will write a KAP/BSB file to file which can be used with most navigation applications and GPS devices. See Section 9.2 for more details.

-K file

This is similar to option **-k** but it generates only a KAP/BSB header instead of a complete file. This is useful if you want to use an external tool such as imgkap to create the KAP file from the PNG.

-l Output page has landscape format rather than portrait, which is default. This option is used only in conjunction with option **-P** if a literal page format is used.

-o file

Set the filename of the output image. *Smrender* parses the string and chooses the output file dependent on the file extensions which can be one of .pdf, .png, or .svg. It defaults to PNG if no valid extension was found. If this option is omitted no image file will be generated.

-O file

Create output PDF file. This option is *deprecated*. See option **-o**.

-p projection

Select the chart projection. This could either be mercator, transversal, or adams2, the 1st being the default value if the options is omitted. Mercator is the typical projection of most nautical charts.

Adams II is a very special projection. It is implemented specifically to create the *Spilhaus World Ocean Map* which was developed by *Athelstan Spilhaus*. See section 6.2.10 for details on how to realize the Spilhaus projection with *Smrender*.

-P fmt|geom[:angle]

Select the page format of the output image. The format can be set to either a named format fmt which has a predefined dimension or as a geometry geom which contains the width and height of the page in millimeters in the format widthxheight.

Fmt currently supports the ISO formats A4 up to A0, and the ANSI formats A to E, letter, legal, and ledger. If this option is omitted the format A3 is selected by default.

If the area to be rendered is specified as bounding box (see Section 4.1) this will most probably colide with the dimension of the page because of the requirements of the projection in which case *Smrender* will resize the bounding box appropriately. If either the width or the height (but never both) is set to zero, *Smrender* will calculate the missing dimension according to the bounding box and the projection.

The parameter angle (prepended by a colon) is optional and by default set to 0. With this the whole page may be rotated. The rotation is defined clockwise in degrees. Internally, the rotation is not applied before saving the final documents.

-m Disable memory mapping (see option **-M**).

-M This option is mandatory if the input file is larger than the amount of memory available on the rendering system. If the system has enough memory this option can be omitted. Using memory mapping probably is a little bit slower but heavily depends on the operating system. On Linux kernel 2.6.32 there is no significant difference in speed.

In any case, files which are larger then 2 GBytes are only supported on 64 bit operating systems.

This option is now on by default.

-N ofs

With this option *Smrender* adds ofs to all nodes and ways IDs as well as to the node references of the ways for all OSM files that are generated by *Smrender* by using the option **-w** or the action out() (see Section 6.3.2).

This options is mainly intended to renumber IDs from input files.

-n Some actions of *Smrender* generate new objects. Negative ids are added to those objects. If this data created by *Smrender* is fed to another OSM tool negative ids may cause problems. With this option all negative ids are output as positive values.

-r file

This option specifies the file name of the rules file. If it is omitted, *Smrender* expects the rules file to be named rules.osm. If file is set to none *Smrender* is run without rules.

If file is a directory *Smrender* will scan it and read all files which match "*osm". The files are read in alphabetical order, simply compared with strcmp(3). Thus, it is suggested to create symbolic links to the files in name it in an appropriate way similar to SysV init scripts.

If you need a more specific order of rule execution you should consider to use the version and/or id fields of the OSM objects (see Section 4).

-R file

This option may be used to save the rules again to another OSM file. This fill will be well-formed in terms of OSM/XML format and may be loaded into JOSM.

-s f

This option scales all images used in the img() function by the factor f (see Section 6.1.3). By default this is 1. This factor is applied additionally to a scaling defined in img().

-S file

With this option all rules are saved to a well readable file in JSON format. All rules are saved in exactly the same order as they are rendered.

The main purpose of this is for debugging of the ruleset. Specifically when writing a ruleset spread over several files one may quickly loose the overview about the order of the rules. Since the order is highly important, this helps you debugging it.

-t title

This option generates a descriptional title node which can be rendered onto the final chart. It is as well used as a title for KAP files (see option **-k**).

-T z ':' dir [':' ftype]

Create tiles suitable for an online viewer into the directory dir. The zoom levels are specified by z which can be either a single value or a range, e.g. 10-15.

The optional parameter ftype sets the file type of the tile images which can be either png (which is default) or jpg.

-u With this option *Smrender* will output a set of URLs which are suitable for OSM data download for the area of rendering specified by the window.

-V With this option set, *Smrender* parses all arguments, and calculates and prints the rendering parameters to stderr. Then it exits immediately.

-v With this option *Smrender* will output the version information of *Smrender* and libcairo to stdout and exit immediately.

-w file

This option specifies the file name of an OSM output file. *Smrender* will dump all nodes, ways, and relations of its internal in-memory database to this file.

During processing and rendering, *Smrender* may generate a lot of additional nodes and ways depending on options and the ruleset, in addition to the input data.

This data will include e.g. the grid but also objects and tags which have been modified or added by the actions found in Section 6.2.

The output file is a valid OSM data file and could be opened in other OSM processing tools such as JOSM.

-x With this option *Smrender* will use an index file to speed up the loading process.

Basically at the startup *Smrender* pulls in the input data file (from option **-i**). By reading it it parses the whole data file and creates an in-memory index which obviously takes some time. Although the XML processor of *Smrender* is pretty fast it is still a time consuming task.

With this option set it will save this internal index to a file after the input data was parsed. In a subsequent run of *Smrender* it will pull in the index directly from the file instead of parsing the whole XML file of the raw input data. This drastically speeds up the initialization process which is very usefull if you render several files in a row if they are based on the same input

data. It is also very useful if you are developing a ruleset and have to restart *Smrender* over and over again, specifically if the input file has a reasonable size.

Before loading the index it compares the timestmaps of the index file and the input file. If the input file is newer it will drop and rebuild the index file. This comparison is based on the *modification time* of the files in the file system. Please note that if you tamper with these timestamps or your system had a timewarp (system time changed) this may lead to accidentally reading an outdated index. In this case just delete the index file manually and restart *Smrender*.

The index file is created in the same directory as the input file as specified by **-i** and its name will have the extension .index added. This location can be changed with option **-X**.

-X This option specifies the name and location of the index file (see option **-x**). This may be desirable if e.g. the OSM input file rests on a read only storage.

If this option is set, option **-x** is implied automatically.

5. Ruleset Definition

The rule set is also defined in OSM format. It contains nodes, ways, and relations together with tags. Nodes are considered to be rules for rendering nodes, ways are rules applied to ways, and relations are rules applied to relations. Each object (node, way, or relation) has a list of tags. These tags represent patterns which are matched against the tags of the objects which are to be rendered (as input by option **-i**). The values of a tag's key (k="...") and/or value (v="...") may be either just a string which is directly matched in a case-sensitive manner or a special match operation (see Section 5.3). The match operations can be used for the key as well as for the value.
Each object has to have a special tag which defines the action that should be carried out in case of a match. This tag has the form _action_=*. The actions are described below in Section 5.4. Of course this tag is not taken into account during the matching process as described previously.
The match algorithm always applies all tags to match, and all of them have to match in order to execute the action. If just a single tag does not match, the object is skipped.

Example: The following rule places a caption with the specifed font at the center of all nodes which have the tags scalerank=2 and FEATURECL=Admin-1* where the latter is matched as a regular expression. The caption is based on the value of the tag NAME.

```
<node>
   <tag k='FEATURECLA' v='/Admin-1.*/'/>
   <tag k='scalerank' v='2'/>
   <tag k='_action_' v='cap:font=sans-serif:bold;size=1.8;key=NAME'/>
</node>
```

5.1. Open and Closed Ways

Ways are a series of nodes. They can be open like lines or closed like polygons (see also Section 6.1.2). Closed ways are such where the first node in the list of nodes is the same as the last node.
By default way-rules match all ways, independently if they are open or closed but there is an option to tell *Smrender* to match either just closed or open ways. This is done by adding two references to the rule with ids which are either equal or unequal to each other. *Smrender* does not care about the absolute value of the reference ids but just its equality.

Example: The following rule writes all closed ways – and only closed ones – with the tag featurecla=Admin-0 to the file admin0.osm. Closed ways are chosen because there are two references which both share the same id (1 in this case).

```
<way>
   <tag k='featurecla' v='Admin-0'/>
   <tag k='_action_' v='out:file=admin0.osm'/>
   <nd ref='1'/>
   <nd ref='1'/>
</way>
```

5.2. Order of Rule Execution

A typical ruleset consists of a set of node, way, and relation rules. It is important to understand the order in which the rules are executed. Since a page is flat and 2-dimensional, rules which are executed later may paint over those that have been executed before.
As defined by the OSM specification[2] all object types (nodes, ways, relations, also called *elements*) have common attributes. Beside the object type, *Smrender* uses the attributes version and id to determine the order of rule execution. All rules of the same version are rendered in the order of their id ascendingly within each type of object. This is repeated for each version ascendingly until the last rule (highest version and highes id). The following enumeration outlines the rendering order.

1. All objects are selected according to their version number, lowest number first.
2. All objects with the same version number are selected according to their type: all relations before ways before nodes.
3. All objects of the same type (and version) are chosen according to their id, lowest id first.

Rules with a version greater or equal to $2^{16} = 0\text{x}10000 = 65536$ are ignored. Numbers equal or greater to that are reserved to be used for sub-rules. See Section 6.3.4 for further details.
The number of iterations, i.e. the number of different versions is limited to MAX_ITER[3] which is defined in rdata.h. Run *Smrender* with option **-V** to see its value.
If the version attribute is missing it is set to 1 by default. All objects without id are numbered ascendingly in the order in which they occur in the rules file starting with some low negative value.
The attribute visible can be set to either true (which is default if omitted) or false. Rules which are "invisible" are not executed. This can be used to enable or disable a rule by default and can further be used for conditional rendering (see enable_rule in Section 6.3.5).
Please note that the initialization function of a rule still is called even if visible=false.

5.3. Match Operations

Basically there are the four different match operations *string compare*, *regular expression match*, *greater than*, and *less than*. Additionally, all of them may be *inverted*, or *excluded*.

[2] http://wiki.openstreetmap.org/wiki/Elements
[3] It is currently (21st of October 2014) defined to 64.

- *String compare* is the most basic match operation. It does an exact case-sensitive string compare. The following tag matches all objects which own the tag seamark:light_character=*.

```
<tag k='seamark:light_character' v=''/>
```

  The empty value v='' represents a wildcard match. It matches any string.

- *Regular expressions* are invoked by enclosing the expression within two slashes /.../ as it is usual in Perl and several other languages. The expression is interpreted as a POSIX extended regular expression (see manpages regex(3) and regex(7)). The following expression matches any object which is tagged with either highway=primary or highway=secondary.

```
<tag k='highway' v='/^(secondary|primary)$/'/>
```

- *Smrender* can interpret tag values as numerical values. Thus, it can do arithmetical comparisons which is *less than* ($<$) and *greater than* ($>$).

  The rule has to contain a number enclosed in square brackets. The direction of the brackets denotes the comparison operation; $[x]$ means that the value of the tag should be lower than x and $]x[$ matches if the tag value is greater than x. The tag value as well as x are always interpreted as decimal number with double precision.

  Square brackets are used deliberately instead of angle brackets to avoid possible conflicts with the XML format or buggy parsers.

  The following rule matches all objects whose seamark:light:range-value is greater than 7.5.

```
<tag k='seamark:light:range' v=']7.5['/>
```

- Match inversion is done by enclosing the match expression within exclamation marks. This inverts the match if and only if the expression without inversion would match. This means that if a tag would match the expression, the inversion would return "false" (no match). But it would not return "true" (match) if the expression would not match.

  The following expression would match all objects which have a tag with the key seamark:type but its value is neither landmark nor light_major.

```
<tag k='seamark:type' v='!/landmark|light_major/!'/>
```

- Match exclusion is denoted by enclosing the match expression within tildes. It does not match objects with certain tags, i.e. it matches the absence of the given tag. The following rule avoids matching of objects which have a tag seamark=*, i.e. all objects match which do not have such a seamark=* tag.

```
<tag k='~seamark~' v=''/>
```

5.4. Rule Actions

Smrender supports a number of powerful built-in actions which are carried out upon successful match. As already mentioned at the beginning of this Section, actions are defined simply with the tag _action_=*. The actions are actually function calls either within the code of *Smrender* or externally from a dynamic library. Thus, there is an unlimited range of extensibility of *Smrender*.

A number of parameters may optionally be passed to the function. The order of the parameters does not matter.
The following example shows an action which places an image at the position of a node.

```
<tag k='_action_' v='img:file=icons/Light_Minor.png'/>
```

The basic format of an action and basic values and types are defined as follows.

```
<action> := <ref> [ ':' <param> [ ';' <param> [ ... ] ] ]
<ref>    := <func> [ '@' <library> ]

<param>  := [ SEP ] <name> [ SEP ] '=' [ SEP ] <value> [ SEP ]
SEP      := ' ' | '"'
<value>  := <bool> | <num> | <coldef> | any valid XML string

<bool>   := 'yes' | 'no' | 'true' | 'false' | <num>
<num>    := any decimal number
<length> := <num> [ <udef> ]
<udef>   := <pageunit> | <nautunit>
<pageunit> := 'mm' | 'cm' | 'in' | 'px' | 'pt' | '%'
<nautunit> := 'degrees' | 'deg' | 'min' |
              'm' | 'km' | 'ft' | 'nm' | 'sm'
```

Every action consists of a symbol name func which is used to find the appropriate function in *Smrender*or in the shared object[4] library, and a set of parameters. These are attribute/value pairs. The names of the attributes are case-sensitive. A special *type* exists which is the boolean type. It can be set to the case-insensitive strings 'yes', 'no', 'true', or 'false', or to any decimal number. 0 is interpreted as false all other values are considered to be *true*.
Smrender is shipped with a set of functions for rendering. Those are capable to place captions (Section 6.1.1), drawing and filling (Section 6.1.2), placing icons (Section 6.1.3), generating OSM files (Section 6.3.2), do some special purpose operations (Section 6.3.5), and calling external library functions (Section 6.3.1). The latter is thought to be a simple but powerful interface for third-party modules.
All functions are described in Section 6.

5.5. Measurement Units

Starting with revision 1851 *Smrender* knows about a set of standard measurement units.[5] There are two groups of units. The first one is related to the page to which the chart is rendered and the second group is related to reallity. Units referred to the page are listed in Table 1 and those referred to reallity are shown in Table 2.
Units are used in combination with a decimal value. As usual, the unit symbols are written behind the number and may be separated with zero or more white spaces.
Pixels do not have a fixed conversion factor. It depends on the pixel density and it is *only* important for raster image output. The density (dpi, dots per inch) is set with the command line option **-d**. The default value is 300.
Please note that *Smrender* internally operates vector-based and the suggested output format is PDF which is created as vector image as well. There are no pixels.

[4] Function libraries, on Unix-based systems .so files, on Mac OSX .dylib files, and on Windows .dll files.
[5] Please note that not all functions already know about that.

Symbol(s)	Descripton	Conversion factor
mm	Millimeters	= 0.03937" = 0.3527 pt
cm	Centimeters	= 10 mm
in or "	Inches	= 25.4 mm = 72 pt = 300 px
px	Pixel	= 0.0846 mm = 0,0033" (see below)
pt	Points	$=\frac{1}{72}$"

Table 1: Measurement units referred to the output page.

Symbol(s)	Descripton	Conversion factor
nm	Nautical miles	= 1852 m = 6076.12 ft
kbl	Cable length	= 185.2 m = 0.1 nm
' or min	Minutes	= 1 nm
° or deg	Degrees	= 60'
m	Meters	= 3,2808 ft
km	Kilometers	= 1000 m
ft	Feet	= 0.3048 m

Table 2: Measurement units referred to reality.

Degrees and minutes are usually (if not specified different) referred to a great circle, i.e. 1° = 60' = 60 nm.

6. Rendering Functions

Generelly, a function does „something“ with the objects that match. The functions are called repeatedly for each objects with matching tags.
It can be distinguished between three types of functions:

1. Functions which directly produce graphics output. Those are also called *rendering primitives* and are described in Section 6.1.

2. Functions which manipulate OSM data, this is modifying or adding OSM elementes to the data. They are described in Section 6.2.

3. Special purpose functions which either control *Smrender* itself or do something else specific, e.g. producing OSM output files. These are described in Section 6.3.

Here is a list of all functions currently implemented and built-in into *Smrender*. Please note that there are additional sea-chart-specific functions found in the the shared library libsmfilter which is bundled with *Smrender*. These functions are described in Section 8.1.

add, 6.2.1
bearings, 6.2.11
boundingbox, 6.2.11
cap, 6.1.1
cat_poly, 6.2.5
check, 6.2.11
clip, 6.2.11
del_match_tags, **??**
diff, 6.3.5
disable, 6.3.5
disable_rule, 6.3.5
dist_median, 6.2.11
draw, 6.1.2
enable, **??**
enable_rule, 6.3.5
exec, 6.3.3
exit, 6.3.5
fixbordernodes, 6.2.11
global_grid, 6.2.11
grid, 6.2.7
grid2, 6.2.7
img, 6.1.3
incomplete, 6.3.5
inherit_tags, 6.2.6
ins_eqdist, 6.2.11
mask, 6.2.11
neighbortile, 6.3.5
out, 6.3.2
poly_area, 6.2.11
poly_centroid, 6.2.11
poly_len, 6.2.11
random, 6.2.11
refine_poly, 6.2.11
reverse_way, 6.2.11
ruler, 6.2.8
ruler2, 6.2.8
set_ccw, 6.2.11
set_cw, 6.2.11
set_tags, 6.2.2
shape, 6.2.3
simplify, 6.2.11
split, 6.2.11
strfmt, 6.2.4
sub, 6.3.4
sync_threads, **??**
transcoord, 6.2.10
translate, 6.2.9
transversal, **??**
virtclosed, 6.2.11
wrapdetect, 6.2.11
zeroway, 6.2.11

6.1. Graphical Rendering Primitives

6.1.1. Captions

The action type cap is used to place a caption. If the action is carried out in a node-rule, the caption is placed at the node's position with the specified properties. The formal definition looks like the following.

```
<action>   := 'cap:' <param> [ ';' <param> ]
<param>    := <fontdef> | <angledef> | <aligndef> | <wcapopt>
              | <fopt> | <hide> | <bground> | <fontbox>
              | <boxscale>

<fontdef>  := <fontname> | <fontsize> | <color> | <fopt>
              | <key>
<fontname> := 'font' '=' <string>
<fontsize> := 'size' '=' decimal value
<color>    := 'color' '=' <coldef>
<coldef>   := <colspec> | '%' <key>
<colspec>  := X11 color name or HTML specification "#xxxxxx".

<key>      := 'key' '=' [ '*' ] <string> | <filter>
<filter>   := '(' <string [ '|' <string> [ ... ] ] ')'
```

```
<angledef> := <angle> | <anglekey>
<angle>    := 'angle' '=' <angleval>
<angleval> := 'auto' | 'majoraxis' | decimal value
<anglekey> := 'anglekey' '=' <string>

<fopt>     := <foptname> '=' decimal value
<foptname> := 'weight' | 'phase' | 'size' | 'xoff' | 'yoff'

<wcapopt>  := <wcapname> '=' decimal value
<wcapname> := 'min_size' | 'max_size' | 'min_area' | 'auto_scale'

<aligndef> := <stat_algn> | <dyn_algn>

<stat_algn>:= <algn> | <halgn> | <valgn>
<algn>     := 'align' '=' <all_alg>
<all_alg>  := <lat_alg> | <lon_alg> | <diag_alg>
<lat_alg>  := 'north' | 'south'
<lon_alg>  := 'east' | 'west'
<diag_alg> := 'northeast' | 'southeast' | 'southwest'
              | 'northwest'
<halgn>    := 'halign' '=' <lon_alg>
<valgn>    := 'valign' '=' <lat_alg>

<dyn_algn> := <dalgn> | <dhalgn> | <dvalgn>
<dalgn>    := 'alignkey' '=' <all_alg>
<dhalgn>   := 'halignkey' '=' <lon_alg>
<dvalgn>   := 'valignkey' '=' <lat_alg>

<hide>     := 'hide' '=' <bool>

<bground>  := 'fillcolor' '=' <coldef>
<boxscale> := 'bgbox_scale' '=' decimal value
<fontbox>  := 'fontbox' '=' <bool>
```

The parameter key is mandatory, the others are optional.
If fontconfig is available, font is defined as specified by fontconfig (see fontconfig documentation). This is e.g. "font=serif:bold". If fontconfig is not available, font must be a full path to a TTF font file.
Size defines the size of the font in millimeters as a decimal value, for example "size=2.4".
Key specifies the key of the tag whose value should be printed. If a caption rule is applied to a node which does not have such a key, the rule simply does nothing. If the key is preceded by an asterisk '*', all letters are capitalized.
The key may be specified directly as a string or as a filter. The filter has to be enclosed in brackets and is a list of strings separated by a '|' charachter. If several strings are given in such a filter expression, the strings are matched against the object in exaclty the order as they appear in the list. Once a string matched, it is used to render the caption.
The main purpose of this is to choose one of several tag candidates dependent if they exist in the

object's list of tags. This may be useful if you want to e.g. render the name of an object in case several name tags exist.
Let's assume you want to place a caption for a place in your language of choice. But what if there is no such tag but another one in a different language? The following rule would render a caption for all nodes tagged as place=island in German, or English if German doesn't exist, or the default name if there is also no English tag.

```
<node>
   <tag k='place' v='island'/>
   <tag k='_action_'
      v='cap:font=sans-serif;size=3;key=(name:de|name:en|name)'/>
</node>
```

Valign and halign specify the alignment of the caption in respect to its center point which is given by the coordinates of the node. There is a horizontal alignment (halign) which could be either east or west and a vertical alignment (valign) which is one of north or south. If no alignment is specified the caption will be centered.
Color defines the color in which the caption should be set. This is either an X11 color preset[6] such as "white", "yellow", "darkgreen" ..., or a color definition similar to the HTML standard. The color may also be set dynamically based on the tag of an object. This is achieved by setting color to the name of the key whose value should be used, prepended by the '%' character. The value of the key is then inpreted exactly as if it would have been set in the rule set. This means it can be any X11 color name or the HTML color representation, as explained below. The parameter fillcolor let's you specify the background of the caption. If it is set, a box with the given color is drawn below the tex. The color presets reflect the colors of traditional sea charts. The HTML-style color definition has the pattern #[aa]rrggbb. The values rr, gg, and bb reflect the RGB values as a hexadecimal number from 00 to ff. Optionally a transparency may be specified with aa. It ranges from 00 (opaque) to 7f which is absolute transparent. The most signifficant bit is always cleared, hence, setting values greater than 7f has no effect.
The parameter fontbox generates a way within the data which is a box around the text. This may be used by subsequent rules to decorate the background of the text in a more powerful manner as with just fillcolor. The fontbox option may be used together with hide since filling would overpaint the text.
xoff and yoff specify an offset which is additionally added if the caption is *not* placed in the center point. The default value is set to size/2. This is because it looks better if the caption does not directly stick to its baseline or the left or right extremety of its bounding box but has a small distance. Sometimes, this is called *padding*.
The angle defines how the caption should be rotated. It is given as usual in trigonometrics which is degrees counterclockwise from 0 to 360 being 0 the regular left-to-right orientation.[7]
Alternatively, the angle may be set to "auto". This causes *Smrender* to try to find an angle in such a way that it does not colide (or at least as little as possible) possible which other objects that have already been rendered. In case of auto-rotation, the additional parameters weight, and phase may be set.
Smrender virtually rotates the caption from 0 to 360 degrees and calculates the color difference between the foreground (the caption) and the background for each angle. It then chooses the angle with the greatest color difference which should be the place where it is best visible. If the angle

[6] http://en.wikipedia.org/wiki/X11_color_names
[7] Please note that this is different to the angle definition of maritime navigation which is degrees clockwise from 0 to 360 being 0 upwards (North).

is between 90 and 270 degrees, *Smrender* automatically flips the caption that it does not read upside-down.
The parameters weight and phase may influence the auto-rotation if specified. The weight is a decimal value between 0 and 1 (1 is default) which allows to weight the angles of 90 plus phase and 270 plus phase less than the others (a phase of 0 is default). This allows to e.g. prefer left-right angles above top-bottom angles. This makes sense because reading left-right is more easy than reading top-bottom.
For example if "weight=0.7" is defined, northerly and southerly test samples are taken into account only with 70% which leads to the fact that the caption tends to be rather east-west aligned.
The parameter 'anglekey' defines a key which can be used if the caption shall be rotated based on the value of a tag. The value of the parameter angle is added additionally. The parameter angle=auto is ignored if a anglekey is set.
If hide is set to yes or any other boolean true value (see Section 5.4) this function is executed as described above except that it finally does not display the caption. This may be useful if auto-rotation is used (angle = auto). In a first step cap() is called with hide = true. Subsequently other rules may be called by using e.g. the tags anglekey or alignkey.

Captions on ways are handled a little bit different from captions on nodes. Actually, captions on ways (polylines) are not supported yet but captions on areas (closed polygons) are very well supported, although it is still in development.
Smrender will calculate the centroid and the area of the polygon. The caption is than placed at the position of the centroid[8]. If the parameter size is omitted or set to 0, the font size is chosen dependent on the square root[9] of the area multiplied by the scaling factor set in the option auto_scale. The resulting font size can further be influenced by the options min_size and max_size. They define the minimum and maximum font sizes even if the calculation by auto_scale would exceed these values. Thus, larger polygons get larger captions and smaller ones get smaller captions.
It may be the case that that captions may clutter the chart if very tiny islands in respect to the page size and map scale get captions as well. This can be avoided by setting the option min_area. The area is a decimal number in square millimeters (mm^2) in respect to the page (not the real area of the island).

6.1.2. Drawing and Filling

The *draw* action is used to draw lines of various styles and fill polygons. This action uses solid colors (with alpha channel) for its operation. It can also be used to fill areas with an image used as a pattern, almost exactly like the action img() (see Section 6.1.3). The difference between image filling with img() and draw() is that draw() supports *directional* filling (see below) which is useful if an object has holes. Directional filling requires that the direction of the ways are correct which unfortunately is not always the case in the original OSM data. It is a more complex operation and thus slightly slower than standard filling.
If you wish to fill an area with an image as pattern use the option file and any of the image options as described in Section 6.1.3.
The following shows the basic rule format.[10]

```
<action>    := 'draw:' <param> [ ';' <param> ... ]
<param>     := <filldef> | <borderdef> | <options>
```

[8] This is similar to what *Osmarender* does.
[9] It was emperical determined that the square root function works pretty well.
[10] Basic BNF parameters are defined in Section 5.4 on page 12.

```
<filldef>   := <color> | <style> | <width> | <dashlen>
<borderdef> := <bcolor> | <bstyle> | <bwidth> | <bdashlen>
<color>     := 'color' '=' <coldef>
<bolor>     := 'bcolor' '=' <coldef>
<style>     := 'style' '=' <styledef>
<styledef>  := 'solid' | 'dashed' | 'dotted' | 'transparent' |
               'pipe' | 'rounddot'
<bstyle>    := 'bstyle' '=' <styledef>
<width>     := 'width' '=' <num>
<bwidth>    := 'bwidth' '=' <num>
<dashlen>   := 'dash' '=' <dashdef>
<dashdef>   := <length> [ ':' <length> ... ]
<options>   := <dirdef> | <opendef> | <curvedef> | <wavedef>
<dirdef>    := 'directional' '=' <bool>
<opendef>   := 'ignore_open' '=' <bool>
<curvedef>  := <curve> | <cfactor>
<curve>     := 'curve' '=' <bool>
<cfactor>   := 'curve_factor' '=' <decval>
<wavedef>   := <wave> | <wlen>
<wave>      := 'wavy' '=' <bool>
<wlen>      := 'wavy_length' '=' <bool>
```

The action behaves a little bit different if it is a polyline (open way, e.g. a river) or a polygon (closed way, i.e. an area, e.g. a lake).
Independently if it is an open or a closed polygon there is always a filled part which is enclosed with a border. The filled part is defined by color, width, and style of which just color is honored for closed polygons. The border part is defined by bcolor, bwidth, and bstyle. The color definition is explained in Section 6.1.1 on page 15.
The arguments to style and bstyle are one of dashed, dotted, solid, transparent, pipe, or rounddot. If a style parameter is omitted, a solid line is drawn.
Rounddot creates a dotted line like dottet but instead of short lines (squares) round dots are drawn.
Pipe is used to create a line style as is used on sea charts for underwater pipes. It is a combination of dotted and dashed.
Smrender allows to specify the length of dashes and the spaces in betweed with the options dash and bdash.
Width and bwidth are given in millimeters. A width of 0 draws the thinnest possible line.
If the boolean parameter curve is set to true, *Smrender* will use Bezier curves to smoothen lines and polygons. Optionally, curve_factor may be set to any decimal value which can be used to modify the curve extremities. A value of 0.0 disables the curving completely. The default value is defined by the macro DIV_PART which is currently set to 0.2.
Smrender is able to generate wavy lines if the boolean parameter wavy is set. On sea charts this line style is used for underwater cables. Please note that *Smrender* internally uses Bezier curves to achieve this. Thus, wavy and curve are mutually exclusive. The wave length can be influenced with the decimal parameter wavy_length. Any known unit (see Section 5.5) may be used. Milimeters (mm) is the default if no unit is specified.
If ignore_open is set to "1", the rule is applied to closed polygons only.
A special fill mode is used if directional is set to "1". Usually, *Smrender* always fills the inner part of a polygon independently of its direction, i.e. if the nodes of the polygon (way) are ordered

clockwise or counterclockwise. This mode is useful if areas of same type (same tags) are enclosed within each other. This may result in unexpected rendering results. The main reason for that is that OSM is just a two-dimensional database. OpenStreetmap provides special tagging facilities to handle such cases, for example multi-poly relations.

A typical application for the *directional* fill mode on sea charts is the rendering of depth contours, in particular if they are filled in shallow inshore areas. *Smrender* takes care on the direction of the polygons.[11] It always fills the portion which is left of the way. Figure 1 shows an example. Shallow water with a depth less than 10 meters is rendered blue, deeper areas are white (transparent). The white area northeast of the islet Radelj is such a 20 meters area which is enclosed by a more shallow area and than again by a deeper area on the west side of this chart detail.

Filling polygons using this mode works only if the polygons are edited correctly, i.e. their direction is correct. Furthermore it is slightly slower than the regular fill mode.

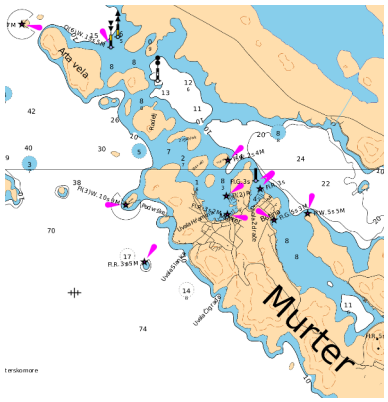


Figure 1: Directional filling.

6.1.3. Placing Images

Smrender allows to place images at the position of nodes or to fill areas using an image as pattern.

```
<action>   := 'img:' [ <param> [ ';' <param> ... ]]
<param>    := <file> | <angle> | <scale> | <mkarea> | <anglekey> | <tr
<file>     := 'file' '=' <string>
<angle>    := 'angle' '=' <num>
<scale>    := 'scale' '=' <num>
<mkarea>   := 'mkarea' '=' <bool>
<anglekey> := 'anglekey' '=' <string>
<trans>    := 'transparency' '=' <num>
```

The parameter file is mandatory and contains a path to a PNG file. If *Smrender* was compiled with librsvg2 (see Section A) it supports SVG files as well. The image is placed with its center directly at the position of the matching node without any modifications.

The parameter angle specifies an angle between 0 and 360 degrees to which to image may be rotated before it is placed onto the map.

If angle is set to "auto", *Smrender* tries to find a rotation angle. This works as described in Section 6.1.1. The rotation function does not take the image itself into account except its size. The rotation test starts at direction East and rotates counterclockwise. Obviously, this makes only sense if it is applied to asymmetric non-centered images, such as light flares. On areas, "auto" has no effect.

The paramter anglekey defines the name of the key of a tag which is used to derive the angle of rotation during runtime. It is implemented in the same way as it is for captions (see Section 6.1.1). Obviously, this does not work together with angle=auto. The latter is ignored if anglekey is set.

The parameter scale allows to scale the image. A value greater than 1 will enlarge the image, if scale is less than 1 it will shrink the image. *Smrender* allows to set a global scaling parameter which is applied to all images, additionally. This is set with the command line parameter **-s** (see Section 4.2).

[11] A few features exist in OSM as well of which the rendering depends on the direction. Most importantly this is natural=coastline. Other examples are waterway=canal and natural=cliff.

The parameter mkarea is a boolean parameter. If set to *yes*, the auto-rotation function will add nodes and ways to the data which indicate the level priority of the angles around the node. This is mainly intended for debugging.
The parameter transparency makes the image transparent. It can be a decimal number between 0.0 and 1.0, where 0.0 is the default value which is absolut opaque (as the original image) and 1.0 is absolut transparent which means you won't see it at all. Please note that this works with pixel images (JPG and PNG) but not with SVGs.

6.2. Data Manipulation

6.2.1. Adding Objects

With this action you can add OSM objects as defined in the ruleset to the data. You can use this to add specific OSM objects during the stage of rendering. Thus, you don't have to "pollute" your source data with map-specific data before. Typically, those objects added are graphically rendered by a subsequent rule. Currently, *Smrender* only OSM nodes are supported.

```
<definition>  := 'add:' [ <param> [ ';' <param> ... ]]
<param>       := <units> | <halign> | <valign> | <reference>
<units>       := 'units' '=' <decval> [ <udef> ]
<decval>      := a decimal number
<halign>      := 'halign' '=' <hdef>
<hdef>        := 'north' | 'south'
<valign>      := 'valign' '=' <vdef>
<vdef>        := 'east' | 'west'
<reference>   := 'reference' '=' <ref>
<ref>         := 'absolute' | 'relative'
```

This function adds a node to the OSM data. All tags of the node are copied except the _action_ tag. The position of the new node is derived from its latitude and longitude. By default, the coordinates are treated as absolute values referred to the geographic coordinate system. if reference is set to relative, the coordinates are treated as relative values to the center of the page. This can be influenced by the parameters halign and valign in which case the origin is set to a page border or page corner. The latitude and the longitude have to be set as decimal number. With the parameter units the unit of measurement can be changed. The default is degrees but mm (milimeters) or cm (centimeters) can be used as well. If relative referencing used, positiv values mean a shift to the right (eastern) and upwards (northern), negative values shift to the left (western) and downwards (southern).

Example The following example adds a node 7 centimeters to the right and above of the lower left page corner.

```
<node lat="70" lon="70">
  <tag k="compass" v="yes"/>
  <tag k="name" v="2°05'E 2003 (5'E)"/>
  <tag k="bearing" v="2.0833"/>
  <tag
    k="_action_"
    v="add:reference=relative;halign=west;valign=south;units=mm"
   />
</node>
```

6.2.2. Adding Tags to Objects

The action set_tags allows to add an arbitrary number of OSM tags to an object. The tags have to be defined through an object within the rules file. This object must not have an action tag. The template should have an id because the action set_tags needs to have a reference to it. The format simply looks like the following.

```
<action>    := 'set_tags:' <param>
<param>     := <name> '=' <value>
<name>      := 'id'
```

Please note that the object type of the rule must be the same as the object type of the template.
Have a look at strfmt in Section 6.2.4 which allows you to dynamically construct and add tags to objects.

Example The following example adds the tags natural=coastline and foo=bar to all ways which have the tags border=page and generator=smrender.

```
<way version="-100" id="115">
   <tag k='natural' v='coastline'/>
   <tag k='foo' v='bar'/>
</way>
<way version="-100">
   <tag k='border' v='page'/>
   <tag k='generator' v='smrender'/>
   <tag k='_action_' v='set_tags:id=115'/>
</way>
```

6.2.3. Standard Shapes

Smrender is able to generate standard shapes like triangles, or circles using this action. The formal definition looks like the following.

```
<action>        := 'shape:' <param>
<param>         := <name> '=' <value>
<name>          := 'nodes' | 'style' | 'radius' |  'angle' |
                   'key' | 'weight' | 'phase' | 'start' |
                   'end' | 'subtype' | 'r2'
```

This action internally generates an ellipse[12] with the given radius (the semi-major axis a) and places a number of nodes on its circumference.
Because OSM does not support any kind of arcs natively, they are constructed using ways with a specific number of nodes. The parameter nodes specifies this number of nodes. Thus, for example, if nodes is set to 3, the result will be a triangle.
The parameter weight is set to 1 by default if it is omitted. It is a multiplier which is used to calculate the semi-minor axis $b = weight \times a$. Thus, if weight = 1 a circle is generated. The parameter phase shifts the points along the circumference in a counterclockwise order. Thus, the following action creates a rectangle of the dimension 4×1.2 millimeters which is rotated by 20 degrees counterclockwise.

[12] Wikipedia: Ellipse, http://en.wikipedia.org/wiki/Ellipse.

```
<tag k='_action_'
     v='shape:nodes=4;radius=2;angle=20;weight=0.3;phase=45'/>
```

One of the parameters nodes or style is mandatory. The latter argument is a preset for a specific number of nodes. Currently the styles triangle (= 3 nodes), square (= 4 nodes), and circle (= maximum nodes) are supported.
The radius is given in millimeters and the optional parameter angle may be used to rotate the shape at any degrees counterclockwise. If radius is omitted 1 millimeter is used as a default value. With key the shape may be rotated dependent on the value of a tag of a node. Key defines the key of this tag.
This action does not render anything itself. It generates an according set of new nodes which are connected together with a way. All these nodes and the way get the tag generator=smrender. The way additionally inherits all tags of the original node which was matched by the rule set to invoke this action. These tags can then be used to render the shape with a way rule. See the following snippet as an example.

```
<node version='-1'>
   <tag k='natural' v='peak'/>
   <tag k='_action_' v='shape:style=triangle;radius=.7'/>
</node>
<way>
   <tag k='natural' v='peak'/>
   <tag k='_action_' v='draw:color=#906030'/>
</way>
```

6.2.4. Create Formatted Strings

This action is intended to create formatted strings out of a set of tags. It works in a similar but yet more simple manner as printf(3) does. The newly constructed string will be added as a new tag to the OSM object. This tag may then be used to match on in subsequent rules.
If the tag already exists in an object it will be overwritten.

```
<action>        := 'strfmt:' <param>
<param>         := <name> '=' <value>
<name>          := 'addtag' | 'format' | 'key'
```

Strfmt() has two mandatory arguments. The first one is addtag which contains the name for the new tag which will be added to this object. The second parameter is format which specifies a format string. It may contain any characters and a set of format symbols. The format symbols are the % character followed by one of the following characters.

- s The string of the value of the tag specified by the respective key is copied to the output string.
- f The string of the value of the tag specified by the respective key is interpreted as a floating point number and copied to the output string.
- d The string of the value of the tag specified by the respective key is interpreted as an integer number and copied to the output string.

[n]r n digits of the fractional part of a floating pointer number is copied to the output string as integer number. The number n is optional. If it is omitted, 1 is assumed.

Example: Let's assume that the original value is 3.1415, then the format specifier '%r' would produce '1' as result, and if '%2r' is specified the result would be '14'.

% The percent character is literally copied to the output string.

v A semicolon is copied to the output string.

All regular characters are directly copied to the output string without conversion. The action *must* contain a key for each format symbol in the format string. The keys are used exactly in the order as they appear in the action line of ruleset.

Format String Example The following example shows how to create a new string for all peaks. It shows the name of the peak and its elevation in parentheses. This string is then rendered in the second rule.

```
<node>
   <tag k='natural' v='peak'/>
   <tag k='_action_' v='strfmt:format=%s (%s);
                  addtag=peak_string;key=name;key=ele'/>
</node>
<node>
   <tag k='peak_string' v=''/>
   <tag k='_action_' v='cap:font=serif;size=2;key=peak_string'/>
</node>
```

6.2.5. Concatenating Ways

```
   <action>   := 'cat_poly:' <param> [ ';' <param> ]
   <param>    := <nocorner> | <incomp> | <copy> | <vcdist>
   <nocorner> := 'no_corner' '=' <bool>
   <incomp>   := 'ign_incomplete' '=' <bool>
   <copy>     := 'copy' '=' <string>
```

This function closes open polygons. To have closed polygons is highly important because just such polygons can be filled.
Polygons which are literally closed, such as the coastline or lakes are very often found as a set of open ways whose beginning and end share the same nodes. This is because different tags may be attached to different parts of the polygon. Furthermore, partial data sets may be used as input because typically just a small area out of the world's data is selected.

Cat_poly has three optional parameters: ign_incomplete, no_corner, and copy. The 1st two are of type boolean. If these parameters are omitted, both are set to 0 (false) by default.
If ign_incomplete is set to 1, cat_poly will only close such polygons which are formed by a collection of ways where the end point of each way directly is the starting point of the next way.
If ign_incomplete is set to 0 (which is the default) cat_poly will also close polygons which are still open even if all ways which have direct neighbors are connected by inserting artificial ways. This is done by connecting the end of each way to the beginning of the next way in the clockwise order of the bearing from the center point of the image to the start/end nodes of the ways.

Please note that the insertion of artificial ways only works properly if the ways have a specific direction. This is true at least for the ways which are tagged with natural=coastline.
The parameter copy can occur multiple times and it is used to specify the keys of the tags of the ways which should be copied to the newly joined longer and closed way. If the values of these keys differ than *Smrender* takes just the first one. All others are ignored. This is because OSM defines that a tag can appear just excatly one time in an OSM object.

If cat_poly is applied in a way rule than *Smrender* tries to close all ways which match the criteria of the rule. The function finds all adjacent ways and closes them properly. The original data is not changed furthermore it creates and inserts new ways. Those new ways are tagged with all tags that where defined in the rule set plus the tag generator=smrender plus all tags which have been specified by the copy parameters.
As already mentioned, a typical application for this function is to close the coastline which will be open in most cases. The coastline is always tagged with natural=coastline.
If cat_poly is applied to a relation then *Smrender* closes all ways of each relation separately. It creates a new closed way which will receive all tags of the relation, respectively. Additionally, it adds the tag generator=smrender. The tags of the way segments are join to the new way if they are listed with the parameter copy as explained above.
Cat_poly applied to relations is useful for example in the Agean Sea, where all partial ways of an island are grouped together using relations. The tags of this relation contains global information about each island, such as its name or population.

Please note that cat_poly() may create ways with more the 2000 nodes which violates the OSM standard definition.[13]. This may cause problems if an output file is created (e.g. with option **-w**) and used in other OSM applications. *Smrender* supports ways of up to 2^{31} nodes. It is assumed that most other OSM processing tools do not care about the artifical boundary of 2000.

6.2.6. Inherit Tags to Objects

```
<action>    := 'inherit_tags:' [ <param> [ ';' <param> ] ]
<param>     := <objdef> | <dirdef> | <force> | <keydef>
<objdef>    := 'object' '=' <obj>
<obj>       := 'node' | 'way' | 'relation'
<dirdef>    := 'direction' '=' <dir>
<dir>       := 'up' | 'down'
<force>     := 'force' '=' <bool>
<keydef>    := 'key' '=' <string>
<string>    := any valid osm key string
```

This action copies tags from objects to their *parent* or children objects depending on the value of direction. A relation is considered as the parent of all of its members and a way is the parent of all of its nodes.
There may be a list of one or more key parameters and optionally the parameters object and force. The keys specify which tags from the child (which is the object to which this rule is applied if direction=up) are copied to its parents – and vice versa if direction=down. If object is not defined, the tags are copied to all parents/children of any type. The parameter object may be set to either way or relation in which only the parents of those types are taken into consideration.

[13] See http://wiki.openstreetmap.org/wiki/Way.

The parameter force may be set to 0 (which is default) or 1. In the latter case inherit_tags will overwrite tags in the parent if they do already exist.

6.2.7. Generating a Grid

```
<action>   := ( 'grid:' | 'grid2:' ) <param> [ ';' <param> ]
<param>    := <margin> | <ticksw> | <sticksw>
              | <grid> | <ticks> | <sticks>
              | <copyrgt> | <cmdline>
              | <gridpts> | <polygon>

<margin>   := 'margin' '=' <length>
<ticksw>   := 'tickswidth' '=' <length>
<sticksw>  := 'subtickswidth' '=' <length>
<grid>     := 'grid' '=' grid spacing in arc minutes
<ticks>    := 'ticks' '=' major axis spacing in arc minutes
<sticks>   := 'subticks' '=' minor axis spacing in arc minutes
<copyrgt>  := 'copyright' '=' <bool>
<cmdline>  := 'cmdline' '=' <bool>
<gridpts>  := 'gridpoints' '=' integer >= 2
<polygon>  := 'polygon_window' '=' <bool>
```

This action creates a grid exactly like the command line option **-g** but it provides more options. It provides the parameters margin, tickswidth, and subtickswith to adjust the size of the axis rulers. Values are given in millimeters. Furthermore it provides the parameters grid, ticks, and subticks which work exactly like the command line option (see Section 4.2). Missing parameters are initialized with default values.
The boolean arguments copyright and cmdline do not generate these nodes.
Each grid line can be generated with a number of intermediate points. The option gridpoints sets the total number of grid points. It has to be greater or equal to 2. If this option is not given or a number smaller than 2 is set, it is internally reset to 2.
The difference between grid() and grid2() is that the 1st generates the grid within the initialization function of the rule while the latter creates it in the main function at the first match. This may be of interest if some rules should be called before the grid is constructed.

6.2.8. Create a Ruler

```
ruler()
```

This function allows to generate a metric ruler. It is thought to be used for land maps. It takes the arguments section and count. Section is the length of one section of the ruler in kilometers. Count sets the number of sections.

6.2.9. Text Translation

```
<action>    := 'translate:' [ <param> [ ';' <param> ] ]
<param>     := <keydef> | <iddef> | <newtagdef> | <coldef>
<keydef>    := 'key' '=' <keyval> [ ';' <keydef ]
```

```
<keyval>    := <string> | '/' <regex> '/'
<iddef>     := 'id' '=' <objid>
<newtagdef> := 'newtag' '=' <bool>
<coldef>    := 'colkey' '=' <regex>
```

This function translates tag values. It can be used to e.g. translate text into different languages. The function looks up a given value in a translation table and replaces its value accordingly. If no entry is found in the translation table, no translation is performed.
The keys whose values should be translated are given with the parameter key. It can be specified several times to be applied to several tags of an object at once. The value of key can be a simple string or a regular expression enclosed in '/'s.
By default the values are replaced in-place. If the old value should be preserved, set the parameter newtag=1. *Smrender* will then insert a new tag into the list of tags. Its value will be the translated value and the name of the key will be appended by the string ':local'.

The translation table is defined in the ruleset as well like a template (see Section 6.2.2). This is a rule node with an action. It has to have a node id because it is referred to by the translate()-rule. This translation object (the template) lists key-value-pairs being the key k the original value and the value v the one translated one.
Have a look at Section 10.2 for a detailed example.

6.2.10. Translate Reference Point of Chart

```
<action>    := 'transcoord:' [ <param> [ ';' <param> ] ]
<param>     := <xdef> | <zdef>
<xdef>      := 'tlat' '=' decimal value
<zdef>      := 'tlon' '=' decimal value
```

This function takes the two arguments tlat and tlon which translates the point of reference. I.e. it rotates the surface of the world.
The values are given in degrees and are used as usual in the navigation. This means that positiv values rotate to the North and East and negative values rotate to the South and West.
Please note that the order of rotation is highly important. This means that the result is different if you rotate 1st longitude then latitude or vice versa. If both parameters tlat and tlon are set, tlon is applied 1st and tlat 2nd. If you need a different order create to separate rules.
This implementation is a spherical rotation. Since the Earth and WGS84 are based on an ellipsoid, this rotation will produced slightly incorrect results.
Further note that a sphere is a 3-dimensional object, thus there are actually 3 rotation axis, x, y, and z, and not just two. The longitude rotation with tlon rotate by the z-axis, which is the axis of the Earth and the latitude tlat rotation corresponds to the x-axis. Although there is no 3rd parameter, a rotation along the y-axis can still be done since they depend on each other on the surface of a sphere. To accomplish a rotation along the y-axis, first rotate x (tlat) up North by 90 degrees, then rotate by z (tlon) with the desired angle, and finally rotate back South by 90 degrees again.
The translation of the reference point is necessary to implement the Spilhaus projection. It rotates the longitude (z) by 66.94970198 degrees to the East, the latitude (x) by 49.56371678 degrees to the South and finally rotates the y-axis by 40.18 degrees. As explained above, because of this rotation by all 3 axis we need several steps for a correct result. The following rules will accomplished this:

```
<!-- transform chart reference point -->
```

```
<!-- 1. rotate longitude -->
<node version='-20'>
   <tag k='_action_' v='transcoord:tlat=0;tlon=66.94970198'/>
</node>
<!-- 2. rotate latitude, but in the opposite direction,
   this is 90 - 49.56371678. -->
<node version='-20'>
   <tag k='_action_' v='transcoord:tlat=40.43628322;tlon=0'/>
</node>
<!-- 3. rotate lon again which now corresponds to the y-axis -->
<node version='-20'>
   <tag k='_action_' v='transcoord:tlat=0;tlon=40.18'/>
</node>
<!-- 4. finally rotate latitude back South by 90 degrees -->
<node version='-20'>
   <tag k='_action_' v='transcoord:tlat=-90;tlon=0'/>
</node>
```

6.2.11. Special Data Manipulation Functions

- bearings

  This functions adds tags containing bearing information to the nodes of a way. Ways are series of nodes connected on after each other. If we look from one node to the next, the next one lies in a specific direction: this is the bearing and is set with the tag smrender:bearing=<brg> where brg is the angle between 0 and 360 degrees.

  Additionally the tag smrender:coursedev=<dev> is added. It contains an angle which states how much the bearing has changed in comparison to the previous segment of the way. Dev is an angle between -180 and 180 degrees.

  The 3rd parameter is smrender:peakdir=<pd> which contains an angle to wich the "peak" between segments (the middle node so to say) points at. This could be used e.g. for putting captions at this points in an appropriate angle to the way.

  All angles are expressed as compass bearings.

- boundingbox

  This function calculates the bounding box of way and adds the tag smrender:bbox=<lat0>,<lon0>,<lat1>,<
  to the way. The coordinats lat0 and lon0 are the left lower (southwest) corner and lat1 and lon1 are the right upper (northeast) corner of the rectangle.

- check

  This function does an integrity check on ways. This is it checks for subsequent duplicate nodes which will be removed from the list of references. If the final way has 0 nodes, the whole way will be removed from the data.

- clip

  This function clips[14] to chart to a given border, i.e. it cuts of the rendered parts at the page border around to chart.

  The optional parameter to clip() is border which takes 4 decimal comma-delimited values. These are the 4 page border values (northern, eastern, southern, western) in millimeters, e.g.

[14] https://en.wikipedia.org/wiki/Clipping_%28computer_graphics%29

border=15,20,15,20. If border is omitted, *Smrender* clips to the default axis borders of the chart.

- dist_median (EXPERIMENTAL)

  This function calculates the median of the length of the edges of a way. It adds the tag smrender:dist_median=*. The value of the tag contains the result of the calculation.

- fixbordernodes

  It turned out that some charting software, at least the in other respects very good chart editing tool Josm,[15] does not handle points at the "edge of the world" properly. These are points with latitudes of exactly ±90 degrees or longitudes of ±180 degrees.

  This function reduces the coordinates of such points by 1 arc second.

- global_grid

  This function adds a global grid to the data, including the arctic circles.

  Parameters: lat_grid, lon_grid, grid, gridpoints

- ins_eqdist (EXPERIMENTAL)

  This function inserts nodes along a way with equal distances. The distance is given with the parameter distance in any unit according to 5.5. If no unit is specified, nautical miles is taken as default. The newly inserted nodes inherit all tags from the way and additionally the tags generator=smrender, and distance=*, and bearing=* are added. The latter two tags are set to the appropriate values. The bearing may be used by the function shape (see Section 6.2.3) to create appropriate rotated shapes.

- mask (EXPERIMENTAL)

  This function allows to mask nodes if there is a dense cluster of nodes in some places on the chart which would make the result to look like just as a blur of color. Sometimes this is also called *uncluttering*.

  The function has the parameter distance which gives the distance of nodes in nautical miles which should be taken into account. This means that if there are more than one node within the given distance, just the most centered one is “chosen”. All other nodes are tagged with smrender:mask=yes. This tag can then be matched on within the ruleset.

- poly_area

  This function calculates the area of closed polygons in nautical square miles. It adds the tag smrender:area=* to the way. The value of the tag contains the area.

- poly_centroid

  This function calculates the centroid of a closed polygon. It then adds a new node at this position. The node will inherit all tags of the polygon. Additionally, the tag smrender:id:way=* is added whose value is set to the ID of the way, respectively.

- poly_len

  This function calculates the length of a polygon in nautical miles. It adds the tag smrender:length=* to the way.

[15] https://josm.openstreetmap.de/

- random

  This function sets a tag with a random number.

```
<action> := 'random:' ['key' '=' <string> ';' ]
                      ['lo' '=' <num> ';']
                      ['hi' '=' <num> ';']
                      ['type' '=' <bool>]
```

  The random number will be between lo and hi. If type is not set or set to false, the random number will be an integer number. Otherwise a random decimal number will be generated.

  The number is set in the tag smrender:random=<num> except the key is set differently with the parameter key.

- set_ccw and set_cw

  Those functions set the direction of a closed way to either clockwise ("cw") or counterclockwise ("ccw").

- simplify

  This function has the mandatory argument dist. It simplifies ways, i.e. it reduced the number of nodes in a way such that the distance between the nodes is greater than dist. By default dist is set in nautical miles.

- split

  This action can be applied to nodes only. It splits all ways at the matching node into two parts. As a result two ways exist: the 1st one ends at the specific node and the 2nd one begins at the same node.

  The function split() has the options boolean argument remsegment. If it is set to true the way will be split at the matching node but the following segment will be removed. As a result there will also be two ways but they are seperated by one segment, the one which follows the matching node in the original way.

- refine_poly (DEPRECATED)

  This function smooths the edges of polygons. It may take the function arguments iteration and deviation. The first defines the number of loops of the iterative refinement process (default = 3). The latter defines the maximum deviation of the original polyline in meters (default = 50). This avoids too high distortion of the polylines.

- reverse_way

  This function always reverses a way independent of its current direction.

- virtclosed

  This function closes ways which are virtually closed. It takes the optional argument dist which defines the maximum distance at which a way is considered as closed. By default it is set to VC_DIST which is defined in smcoast.h (which is 0.1 meters).

  The function calculates the distance between the 1st and the last node of a way. If this distance is lower than dist it closes the way by appending the first node to the end of the way.

- wrapdetect

  This function detects if a way wraps from East to West or from West to East across the 180th degree Meridian (West or East).

  If such a wrapping is detected it adds the tag smrender:wrapdetect=split to the node before the wrap. This could be used in a consecutive step to call the function split().

  This may be desireable because a lot of chart software (such as *Josm* and *Smrender* itself) cannot handle such situations properly.

  The ruleset could look like the following:

```
<way version='-15'>
   <tag k='_action_' v='wrapdetect'/>
</way>

<node version='-15' visible='true'>
   <tag k='smrender:wrapdetect' v='split'/>
   <tag k='_action_' v='split:remsegment=1'/>
</node>
```

- zeroway (EXPERIMENTAL)

  Insert an artificial way of length zero at the matching node if the node is connecting two other ways. Such a way is a way with two nodes where both nodes have the same position.

6.3. Special Purpose Functions

6.3.1. Calling External Functions

Smrender has the ability to call user-defined library functions. This feature provides modularity and the flexibility to be extended on the fly without modifying the core. Thus, *Smrender* can be used for nearly every kind of rule-based OSM file processing. The library calls dlopen(3) and dlsym(3) are used to dynamically import those functions.
The basic rule format is defined in the following.

```
<definition>  := <function> '@' <library> [':' <param>
                   [ ';' <param> ... ]]
<library>     := path/name of shared library
<param>       := <name> '=' <value>
```

Function is the name of the function as it is exported by the shared object library. In particular, the exported symbol has to be named act_function_main(), i.e. it has to be prefixed by "act_" and suffixed by "_main". If library contains a '/', the path is resolved and the shared object loaded from that location. Otherwise the dynamic linker tries to find the library in the appropriate system directories.[16]
Beside linking the function itself, *Smrender* tries to import the optional functions act_function_ini() and act_function_fini(). These two functions may be used for initialization and finalization of the main function.

[16] See dlopen(3) for details.

The function is called on each match of an OSM node. The initialization function act_function_ini() is called once directly after the rules file was parsed before the first match. The finalization function act_function_fini() is called onced directly after the last match.
The prototypes are defined as follows.

```
int (*act_function_ini)(smrule_t*);
int (*act_function_main)(smrule_t*, osm_obj_t*);
int (*act_function_fini)(smrule_t*);
```

Act_function_main() gets a pointer to the rule structure and the OSM object which matched the rule. The object can be either a *node*, a *way*, or a *relation*.

```
typedef struct smrule smrule_t;
typedef struct action action_t;

struct smrule
{
   osm_obj_t *oo;
   void *data;        // arbitrary data
   action_t *act;
};

char *get_param(const char*, double*, const action_t*);
char *get_parami(const char*, int*, const action_t*);
```

The rule structure contains three pointers. The first one points to the OSM object of the rule as defined in the ruleset. The _action_ tag was removed by the rules parser. The Second pointer is initialized to NULL by *Smrender* and is not touched any further. It is thought to be used by the external functions to store arbitrary data. Please note that all resources that have been claimed by the _ini() function (such as heap memory) have to be freed again by the finalization function _fini(). The third pointer of type action_t contains all data which needs *Smrender* for rule processing. Its contents should not be touched except you know what you are doing. It is important that the action structure (action_t) contains the parameters which may have been passed to the function through the ruleset. The function get_param() shall be used to retrieve their values. The first parameter is a constant string to the name of the parameter. The second parameter is a pointer to a double variable which will receive the converted value of the parameter. Of course this works only if the parameter contains a decimal value. This pointer may be set to NULL if it is not used. The third parameter to get_param() is a pointer to the action structure of the rule.

```
typedef struct osm_obj
{
   // type of object: {OSM_NODE, OSM_WAY, OSM_REL}
   short type;
   // visibility: {0, 1}
   short vis;
   // OSM id
   int64_t id;
   // version, changeset, user id
   int ver, cs, uid;
   // Unix timestamp
   time_t tim;
   // number of tags
   short tag_cnt;
   // Pointer to tags
   struct otag *otag;
} osm_obj_t;
```

The type of object can be determined on examination of osm_obj_t.type. The variable may be set to either of OSM_NODE, OSM_WAY, or OSM_REL. The object can then be type-casted

to either a osm_node_t, a osm_way_t, or a osm_rel_t. All those OSM types are defined in osm_inplace.h.
The return value of the function controls the further behavior of *Smrender* while applying this same rule. A return value of 0 means no error. *Smrender* will call the function again at the next matching object. If the return value is greater than 0 it behaves similar but outputs a message in the log file. The message contains the return value. If a negative value is returned, *Smrender* immediately stops applying this rule, calls the _fini() function and processes the next rule.
Section B explains how to write own (rendering) functions more in detail.

Security Implications This feature basically allows any user to call arbitrary functions on the system. Thus, *Smrender* should **never ever** be installed with file modes SUID/GUID-root! This would be a potential security risk and might allow an attacker with access to your system to compromise it.

6.3.2. Output of OSM Data

The action out outputs all matched OSM objects to a file. The default file format is OSM but with the option format=json it is possible to output it in JSON format as well and format=js it outputs the data formated as Javascript file suitable to be included in a web page or node program. This action has one mandatory option which is file to choose the ouput file name. If several out rules with the same file name exist, all objects are collected into that single file. As shown below there are some additional options but these are honored only in the first occurance of several rules which share the same file name.

```
<definition>  := 'out:' [ <param> [ ';' <param> ... ]]
<param>       := <file> | <format> | <renumber> | <jsonopts> | <precision
<jsonopts>    := <condensed> | <short> | <visible> | <nindent> |
                 <compact>
<file>        := 'file' '=' <filename>
<format>      := 'format' '=' <type>
<renumber>    := 'renumber' '=' <bool>
<condensed>   := 'condensed' '=' <bool>
<short>       := 'short' '=' <bool>
<visible>     := 'visible' '=' <bool>
<nindent>     := 'nindent' '=' <num>
<precision>   := 'precision' '=' <num>
<compact>     := 'compact' '=' <bool>
<type>        := 'osm' | 'json' | 'js'
```

With renumber enabled all objects are renumbered ascendingly starting with id=1. This may be useful and ease data handling in other applications.
The options condensed, short, visible, and nindent apply only to the JSON format. Condensed ommits all spaces and new lines which makes it more dense. It saves a lot of memory which is useful if the data shall be sent over to e.g. a web client. Short uses an even more compact format. It abbrevates the keys of the object structure to just a single character. Visible outputs only visible objects and ommits the visible attribute. This compacts the data even more. Nindent selects the default number of spaces to indent the data if it is output in non-condensed mode. The coordinates of nodes are output as decimal numbers with a precision of 7 by default. This can be changed with parameter precision. It should be an integer number between 0 and 20.
There are currently three different output formats.

- **OSM:** This is the traditional XML/OSM format containing nodes, ways, and relations in that order. This files are well-formed and can be read in any other OSM software (such as e.g. Josm).
- **JSON:** This is a translation of the classic OSM format into JSON containing three arrays which hold all nodes, ways, and relations.

  The argument compact=yes modifies the format into much more dense version. In this case the ways contain not references to nodes (as they do in the original format) but directly the coordinates of the nodes. In contrast, nodes are only found in the array of nodes if they own at least one tag.
- **JS:** This format is almost the same as JSON but it is suitable to be directly included in Javascript either e.g. with a script tag from HTML or with require() in a node program.

6.3.3. Executing Programs and Scripts

Smrender is able to run external 3rd-party programs and scripts. *Smrender* communicates through stdin, stdout, and stderr of the program. The new process is executed by the system call execvp(3) or execvpe(3) if available.

```
<action>        := 'exec:' <param>
<param>         := <name> '=' <value>
<name>          := 'cmd' | 'arg' | 'env' | 'osmhdr'
```

The mandatory parameter cmd defines the path to program. Optionally, one or more arguments can be passed to the program by multiply specifying arg. The arguments are passed exactly in the same order as in the action.
The optional parameter env can be used to set environmant variables. By default, the program is executed with an empty environment. *Smrender* provides an interactive interface to communicate with the process.
The parameter osmhdr is an optional boolean argument and influences the communication protocol as explained in the following Section.

The Communication Protocol is an interactive hybrid command line protocol. All information from *Smrender* to the process is sent in XML format. In turn for simplicity, the process can use simple commands.
The communication is initiated by *Smrender* with an XML header and the *Smrender* header. The latter contains the version of the protocol and the a string for the XML generator, similar to the OSM format.

```
<?xml version='1.0' encoding='UTF-8'?>
<smrender version='0.1' generator='smrender 3.0.r1535'>
```

After this header all OSM objects which machted the rule are sent, one after the other in OSM format version 0.6. If the action has the parameter osmhdr set, each object is included in an OSM header as well. The following shows an example of an object including the OSM header, i.e. osmhdr=yes.

```
<osm version='0.6' generator='smrender'>
<node id="39273652" version="5" timestamp="2009-02-05T06:45:28Z"
```

```
   uid="67265" visible="true" lat="44.2128480" lon="15.4482687">
<tag k="created_by" v="Merkaartor 0.13"/>
</node>
</osm>
```

If osmhdr is omitted or set to no, the first and the last line of the above stanza are suppressed. After each OSM object, *Smrender* waits for commands of the process. Every command will generate some output and finally send a status code. The status code may be used by the process determine if a command could be executed successfully.

```
<status code="200">OK</status>
```

The following commands are currently implemented:

- . [<code>]

  A single period on a line will provoke *Smrender* to send the next object which matches the rule. Optionally a numeric code between -128 and 127 can be supplied. 0 is default (if omitted) and simply means success. *Smrender* appends the status code 200 to each object. If no more objects are available, status code 404 is sent. In that case *Smrender* waits for a last confirmation through a single period.

  A positiv number indicates an error. *Smrender* will stop further processing of objects of this rule. The process has the chance to finalize and has to commit this with a single period again. *Smrender* will close all streams and continue executing the next rule.

  If A negative number is returned, *Smrender* interpretes this as a fatal error. As a consequence it will imediately stop further processing.

- get (node|way|relation) <id>

  Additional OSM objects can be retrieved with this command.

6.3.4. Calling Sub-Rules

The ruleset of *Smrender* allows to call a set of sub-rules. The purpose of this is to make the ruleset more simple and modular. Sub-rules can be nested and are re-entrant. That means that you can call a further subset from within a subset and that a subset can be called multiple times (like function calls in a programming language).
The subroutine call is implemented as a regular action with the following syntax.

```
<action>   := 'cap:' <param> [ ';' <param> ]
<param>    := <version>
<version>  := 'version' '=' integer number
```

The option version is mandatory. It specifies the version number of the group of rules to be called. All rules with this version number and the same object type (node, way, or relation) are then called in the order of their id ascendingly. These rules are then applied to the all objects which match the criteria defined in the sub-rule definition *and* the sub-rules themselves. Have a look at the following example.

```
<!-- call subrules for objects matching these tags -->
<way> <!-- rule #1 -->
   <tag k='natural' v='coastline'/>
```

```
    <tag k='generator' v='smrender'/>
    <tag k='_action_' v='sub:version=65536'/>
</way>

<!-- subrules to be called -->
<way version="65536"> <!-- rule #2 -->
    <tag k='_action_'
    v='draw:color=navajowhite;directional=1;ignore_open=1'/>
</way>

<!-- nested subroutine call -->
<way version="65536"> <!-- rule #3 -->
    <tag k='_action_' v='sub:version=65537'/>
</way>

<!-- next level subroutines -->
<way version="65537"> <!-- rule #4 -->
    <tag k='_action_' v='draw:bcolor=black'/>
</way>

<!-- repeated subroutine call-->
<way version="2"> <!-- rule #5 -->
    <tag k='seamark:type' v='depth_contour'/>
    <tag k='seamark:depth_contour:depth' v='10'/>
    <tag k='_action_' v='sub:version=65537'/>
</way>
```

This example contains 5 rules in 4 groups. The 1st one has no version attribute thus 1 is assumed. The 2nd and 3rd one are in group #65536, the 4th is in group #65537 and the 5th is in group #2. The 1st rule to be executed is the 1st at the beginning because it has the lowest version (group) number and the lowest id – obviously because it is the only element in the group. The 2nd, 3rd, and 4th rule are ignored by the regular rule processor because their version number is $\geq$ 65536 (see Section 5.2). The 5th rule is executed next after the 1st one.
The execution is then done as follows. The 1st rule is executed and applied to all ways which have the tags natural=coastline and generator=smrender. Since the action is sub(version=65536), "applying" means executing all rules from the given subset. This subset contains 2 rules: the 2nd and the 3rd from the ruleset from above. The 2nd is executed 1st because it has the lower id. This is because both rules have no explicitly defined id so they are numbered with an internal number ascendingly as they appear. The 3rd is executed directly after #2. Since this rule is again a subroutine call to the group #65537, rule #4 is executed.

6.3.5. Special Control Functions

- diff (EXPERIMENTAL)

  This function takes the argument file (exactly like in the function out. See Section 6.3.2.) and infile. It compares the ids of all objects in infile with all objects wich have been loaded by *Smrender* (option **-i**) and writes all objects which do not exist to file.

- disable

This function disables the object to which it is applied. This is done by setting the attribute visible="false". It is intended to be used to completely ignore certain objects during the rendering process. Please note that disabled objects cannot be re-enabled again because the rendering engine completely ignores invisible objects.

- enable_rule and disable_rule

  These actions take the mandatory argument id which specifies the id of the rule to be enabled or disabled. They actually set the visibility to either true or false (see Section 5). Please note that enabling a rule which would have been rendered before this one according to its version and id is not executed it again.

- exit

  This function forces *Smrender* to stop. No further rules will be processed and *Smrender* will create the output files according to the command line options. It behaves exactly like sending the INT signal (pressing ^C, see Section 7).

  This function is mainly intended for debugging rule sets.

- incomplete (EXPERIMENTAL)

  This action can be applied to relations andy ways. It takes the single parameter file which specifies the name of the output file to which it will write the types and ids of all objects which are listed as members of the relation, or nodes which are referred to by a way, but are not found in the input data. This may be useful to download these objects to complete the input data if necessary.

  Please note that this currently may not work properly if you specify the same file name in several incomplete-rules.

- neighbortile (EXPERIMENTAL) This is a special function which may be handy in the tile creation process. It creates a directory named "neighbor_tiles" and within this a tile directory tree (zoom/x/y.conf). The files created within this will contain several variables suitable for being sourced in a shell script. The variables in the files specifies the bounding boxes of the tiles in zoom level 10. This function will be improved in future and the file format may change.

7. Signals

Smrender installs two signal handlers, one for SIGUSR1 and one for SIGINT.
If *Smrender* receives a USR1 signal during the process of reading OSM input data, it outputs some statistics about the current position of reading and data throughput. It may be used as progress indicator if huge files are used as input. If *Smrender* receives a INT signal (which is typically generated by pressing ^C) during rendering, it immediately aborts rendering of further objects and saves the image in its current state and exits normally. If SIGINT is caught twice, *Smrender* exits immediately.

8. Extensions

As explained in Section 6.3.1, *Smrender* is able to call functions of shared objects through dynamic linking at runtime. Thus it is very easy to extend the core functionality of *Smrender*. Currently, it comes with one additional library which is libsmfilter.

8.1. Libsmfilter and Smfilter

Smfilter[17] is a preprocessor for Osmarender. It adds some sea chart specific virtual nodes and ways to simplify the rendering process. The functionality of smfilter is now integrated into *Smrender* with libsmfilter. As a result, smfilter is no longer supported. Libsmfilter exports four functions: vsector(), pchar(), compass(), and sounding(). The first is the replacement for smfilter, the next is a new function which generates combined strings for light descriptions, the third creates a magnetic variation compass, and the last generates nodes and ways for depth soundings as used in sea charts.
All functions belong to the category *data manipulation* (see Section 6.2), which add OSM objects to the data. Thus, graphical rendering rules (see Section 6.1) are necessary to display those objects on the page.

8.1.1. Generating Light Sectors with vsector()

This function is a full replacement for smfilter. Smfilter took several options[18] to adjust the rendering behavior. These are the options **-a**, **-b**, **-d**, and **-r** in particular. Libsmfilter now takes exactly the same parameters since it is just a port. The parameters must be fed to it within the rules file. This was commonly described in Section 6.3.1. In detail the format looks like the following.

```
<param-str>    := <a-v-pair>[';' <a-v-pair>[';' ...]]
<a-v-pair>     := <attribute> '=' <value>
<attribute>    := 'a' | 'b' | 'd' | 'r' | 'f'
<value>        := decimal number
```

a This sets the maximum distance of arc nodes. Basically, the distance is scaled with the radius; the smaller the radius the smaller the distance and vice versa. With large radii the distance of nodes is limited to dist. The value is given in nautical miles.

b Leading and directional lights are rendered with a bearing line and a small arc at its end. deg sets the angle of this arc to one side which means it is drawn def degrees clockwise and def degress counterclockwise from the bearing line.

d This defines the arc divisor which is used to determine the distance of the arc nodes. Thus, the node distance equals the radius divided by div. (see also option -a).

r A light may not have any radius specified since the tag is optional. In such cases smfilter picks radius as default value.

f This is a scaling factor for all radii. In particular, this can be useful for charts in very large scales. The default value is 1.

This is an example for calling vsector() from the rule set.

```
<node>
   <tag k='seamark:type' v=''/>
   <tag
      k='_action_'
      v='vsector@libsmfilter.so:a=0.05;d=20;r=0.5'
   />
</node>
```

[17] See http://www.abenteuerland.at/smfilter/
[18] See http://www.abenteuerland.at/smfilter/smfilter.html

A full description of the output produced by vsector() is found in the smfilter(1) man page[19] and in the OSM wiki.[20]

8.1.2. Compatibility to Smfilter

The function vsector() does exactly the same as the original smfilter tool. Thus, they are considered to be nearly 100% compatible. The functionality is exactly the same but the file structure will still be different because *Smrender* processes the OSM file in a different way than smfilter.
The following shows two exchangable command lines, the first for smfilter and the second for *Smrender*.

```
smfilter -a 0.05 -d 20 -r 0.5 < in.osm > out.osm
```

```
smrender -i in.osm -o /dev/null -M -G -w out.osm
```

The following rules file has to be used in conjunction with *Smrender* to be a replacement for smfilter. Of course, the file may be extended with other rendering rules.

```
<?xml version='1.0' encoding='UTF-8'?>
<osm version='0.6'>
   <node>
      <tag k='seamark:type' v=''/>
      <tag k='_action_'
           v='func:vsector@./libsmfilter.so?a=0.05,d=20,r=0.5'/>
   </node>
</osm>
```

8.1.3. Generating Light Description Strings

Pchar() generates a string which contains the characteristics of the light as it is used in official sea charts and the *List of Lights*. See *INT1/Chart No. 1* Section P and P.16 in particular.[21]

```
<action> := 'pchar@libsmfilter.so:' [ <param> [ ';' <param> ]]
<param>  := <lang> | <compat> | <colkey>
<lang>   := 'lang' '=' ( 'de' | 'en' | 'gr' | 'hr' )
<compat> := 'compat' '=' <bool>
<colkey> := 'colkey' '=' regular expression
```

The function analyzes the tags of an object. If it contains valid *OpenSeamap* tags[22] it generates the light string and adds the new OSM tag seamark:ligh_character=* to the object. The value of the tag contains the string which may be rendered by a subsequent rule.
Since the description of lights varies in different languages, *Smrender* supports the optional parameter lang having 'en' as default. It depends on the compatibility setting of 'compat' how it will behave.
By default it is set to compat=yes for backwards compatibility. In this case it will 1st translate the tags into the chosen language and then format it according to the local *INT1*.

[19] http://www.abenteuerland.at/smfilter/smfilter.html
[20] http://wiki.openstreetmap.org/wiki/OpenSeaMap/smfilter
[21] http://www.nauticalcharts.noaa.gov/mcd/chart1/ChartNo1.pdf
[22] See http://wiki.openstreetmap.org/wiki/OpenSeaMap/Lights_Data_Model.

Please note that this mode is deprecated since adding new languages would require to change the code the new suggested variant is to set compat=no. In this case it formats the string according the chosen language's *INT1* but it does not translate the light characters and colors. This must be done beforehand by using the action translate() (see 6.2.9).
The following table shows language examples according to *INT-1, IP 51*:

English	Fl.10s 40m 27M
German	Blz.10s 40m 27sm
Croation	B Bl 10s 40m 27M

8.1.4. Generating a Magnetic Variation Compass

This function creates a magnetic variation compass as usual on sea charts. This is a north-up compass with with crosslines rotated by the variation as shown in Fig. 2. As most other functions, it will not directly create graphics output but OSM nodes and ways with specific tags. Those can then be graphically rendered by using one of *Smrender*'s rendering primitives (s. Section 6.1.1 and 6.1.2).

```
<action> := 'compass@libsmfilter.so:' <param> [ ';' <param> ]
<param>  := <var> | <radius> | <ticks> | <stldef>
<var>    := 'variation' '=' decimal value
<radius> := 'radius' '=' decimal value
<ticks>  := 'ticks' '=' integer value
<stldef> := 'style' '=' ( 'int1' | 'int1var' )
```

The parameter radius is mandatory and defines the outer radius of the compass. It is given as degrees on a Meridian.[23]
The parameter variation sets the variation, i.e. the amount of degress to which the crosslines will be rotated. It is set in degrees. A positive value denotes eastern variations.
The parameter ticks defines the number of ticks on the circle. The default value is 360.

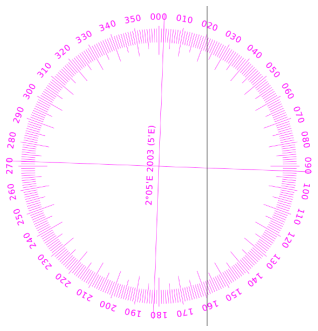


Figure 2: Magnetic variation.

All newly created nodes and ways are tagged with smrender:compass=* containig the bearing of the way in degrees. Every tenth outer node is additionally tagged with smrender:compass:description=* containing the bearing in degrees as well but with a leading zero as usual for the description of degrees on a sea chart.
See Section 10.1 for a complete example.

8.1.5. Generating Circles around Depth Soundings

Libsmfilter provides the function sounding() which generates small circles around depth soundings.
It generates symbols such as I.4 of Chart No. 1 and circles with a dashed line used for approximate depths (I.31).

[23] This is subject to being improved in future releases.

Tags use for it is seamark:sounding=* containing the depth in meters, and optional seamark:sounding:quality = {approx | reported_unconfirmed}. See "Rendering Depths with *Smrender*"[24] for more details.

9. Usage Examples

There is a very simple example for your first rendered map. Before, download, compile, and install *Smrender* as explained in Section A.
Create a working directory somewhere and change into it.

```
mkdir mychart
cd mychart
```

Now symlink the chart symbols which are included in the installation package and copy one of the default rulesets.

```
ln -s /usr/local/share/smrender/SeaMapSymbols
mkdir rules.osm
cp /usr/local/share/smrender/r100k_ye/* rules.osm
```

Now we have to get some OSM data. We just use the Overpass API[25] to get a small window.

```
wget -O cr.osm \
   'http://www.overpass-api.de/api/xapi?map?bbox=15,43.7,15.4,44'
```

Now we can run *Smrender* as explained in the following sections.

9.1. Generating a PNG or PDF

PDF files are the best choice if you intend to print the result. The following command line renders the OSM file cr.osm to the output image cr.png having the dimension of an A4 landscape page. 43° N 52.8' 015° E 12.8' are the center coordinates and the scale is chosen to be 1:100000 which is typical for sea charts. Option **–s** 0.33 scales all icons by 0.33.

```
smrender -i cr.osm -o cr.png -P A4 -l -s 0.33 43N52.8:15E12.8:100000
```

The result is the PNG image cr.png. You may look at it using your favorite image viewer.
To get a correct non-distorted print-out it usually is a good idea to use the PDF format instead. If you try to print the image with a graphics program it most probably will be rescaled to fit the print margins. This will usually not happen if you print a PDF which has valid paper dimensions.
To create a PDF file just used `.pdf` as extension for the output filename. You could specifiy the option **–o** to create several output files at once.

```
smrender -i cr.osm -o cr.pdf -P A4 -l -s 0.33 43N52.8:15E12.8:100000
```

[24] http://www.cypherpunk.at/2012/03/11/rendering-depths-with-smrender/
[25] http://wiki.openstreetmap.org/wiki/Overpass_API.

9.2. Generating a KAP File

KAP files are used by many applications that deal with marine navigation (e.g. *OpenCPN*, `http://opencpn.org/`) or GPS chart plotters or smart phones (e.g. *Marine Navigator* for Android, `https://play.google.com/store/apps/details?id=de.kemiro.marinenavigator`). Very often those files are also referred to as BSB files or also RNC files.
The following command generates a KAP file. It assumes that you have an input OSM file cr.osm. See the beginning of this Section (Section 9) on how to retrieve it. The density is reduced to 200 dpi (option **-d**) to save resources of your smart phone or chart plotter. Option **-G** disables the grid since most applications are able to generate a grid on their own. The KAP file is saved to cr.kap.

```
smrender -i cr.osm -k cr.kap -s 0.33 -G -d 200 43N52.8:15E12.8:100000
```

Please note that a PNG file, a PDF file, and a KAP file may be generated at the same time by simply adding option **-o**. The file cr.kap can be used by your favorite application. If you use e.g. *Marine Navigator* on Android you have to copy the file to a folder named BSB_ROOT in the root directory of your SD card.

10. Ruleset Examples

10.1. Creating a Magnetic Variation Compass

```
<node lat="70" lon="70" version="-10">
   <tag k="compass" v="yes"/>
   <tag k="name" v="2°05'E 2003 (5'E)"/>
   <tag k="bearing" v="2.0833"/>
   <tag
    k="_action_"
    v="add:reference=relative;halign=west;valign=south;units=mm"
    />
</node>
<node version="-10">
   <tag k="compass" v="yes"/>
   <tag
    k="_action_"
    v="compass@libsmfilter.so:radius=0.03;variation=2.0833"
    />
</node>
<node>
   <tag k="compass" v="yes"/>
   <tag
    k='_action_'
    v='cap:font=sans-serif;color=magenta;size=2;key=name;
          anglekey=bearing;angle=-270;valign=north'
    />
</node>
<node>
   <tag k="smrender:compass:description" v=""/>
   <tag
    k='_action_'
    v='cap:font=sans-serif;color=magenta;size=2;
          key=smrender:compass:description;
          anglekey=smrender:compass;angle=0;valign=north'
```

```
    />
</node>
<way>
   <tag k="smrender:compass" v=""/>
   <tag k="_action_" v="draw:bcolor=magenta;width=0.2"/>
</way>
```

10.2. Translation Example

The following paragraphs demonstrate a translation of tags. First we need a translation table. The following shows a translation table for translating colors from English to German.

```
<node id='1000'>
   <tag k='red' v='rot'/>
   <tag k='green' v='grün'/>
   <tag k='blue' v='blau'/>
   <tag k='yellow' v='gelb'/>
</node>
```

Now we want to apply the translation to seamark nodes which have English names for the color of the lights. The name of these tags are either 'seamark:light:colour=*' or 'seamark:light:<N>:colour=*' where <N> is any integer greater than 0. A node can have multiple such tags. The rule should then look like this:

```
<node>
   <tag k='/seamark:light:.*colour/' v=''/>
   <tag
      k='_action_'
      v='translate:id=1000;key=/seamark:light:.*colour/;newtag=1'
   />
</node>
```

Let's look at the following node which could be within the input data.

```
<node id='12345678' version='1' lat='47.1234' lon='10.2323'>
   <tag k='name' v='Red-White-Lighthouse'/>
   <tag k='seamark:light:1:colour' v='yellow'/>
   <tag k='seamark:light:2:colour' v='red'/>
   <tag k='seamark:type' v='beacon_lateral'/>
</node>
```

Smrender will modify it according to the translation rule of above like follows.

```
<node id='12345678' version='1' lat='47.1234' lon='10.2323'>
   <tag k='name' v='Red-White-Lighthouse'/>
   <tag k='seamark:light:1:colour' v='yellow'/>
   <tag k='seamark:light:1:colour:local' v='gelb'/>
   <tag k='seamark:light:2:colour' v='red'/>
   <tag k='seamark:light:2:colour:local' v='rot'/>
   <tag k='seamark:type' v='beacon_lateral'/>
</node>
```

11. Files

The *Smrender* package contains all source C files and headers. A configure script is provided to create appropriate Makefiles and build *Smrender* (see Section A). It contains all sources for the smfilter library (see Section 8.1) in the directory libsmfilter/ and a skeleton libary in the directory libskel/ which may be used as a starting point for own functions (see Section B). Due to an internal code reorganization many general purpose functions are moved to the separate library libsmrender. All respective sources are found in the directory libsmrender/.
The package contains futhermore different rule sets which may also be used as a basis for own rule sets. The main ruleset is found in the directory rules_100000 which is actively maintained. Older files are rules.osm, rulesbig.osm, and rules_land.osm which are still provided with *Smrender*.

12. Bugs and Caveats

Smrender does not validate the well-formedness of the OSM files. Thus, you may get unexpected rendering results if the file format is incorrect.

13. Author

Smrender is written by Bernhard R. Fischer, `mailto:bf@abenteuerland.at`. The idea of the project was born in summer of 2010. The actual development started in October of 2011.

14. Copyright

Copyright 2011-2022 Bernhard R. Fischer.

This file is part of *Smrender*.

Smrender is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, version 3 of the License.

Smrender is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with *Smrender*. If not, see `<http://www.gnu.org/licenses/>`.

A. Compiling and Installing

Smrender should be simple to compile. It depends on libcairo[26] and optionally on fontconfig and librsvg2. Although they are not mandatory it is still suggested to use those packages. With fontconfig you can specify simple font faces such as e.g. sans-serif or DejaVuSans and the package will handle the path resolution to the necessary font files. You can still use fonts even without fontconfig but you then have to specify the full paths for the font files in the cap() rules (see Section 6.1.1) such as e.g. /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf. The package librsvg2 is used to read SVG images in your ruleset, otherwise it supports just PNG images (see Section 6.1.3).
If none of those packages is not installed, *Smrender* will still compile and it can be used for OSM data processing but not for rendering charts (graphics output).
Download the most recent *Smrender* package from its main URL.[27] The packages includes the source code, some sea chart rulesets and a few ruleset examples, and a skelleton library (libskel) to show how to implement an own library with extension functions. It further includes libsmfilter for rendering special sea chart features (see Section 8.1), and a complete set of SVG icons for sea charts (buoys, beacons,...).

Extract the package with xzcat | tar xv smrender-4.0.r$xxxx$.xz. Change into the newly extracted directory. Then run the configure script ./configure, then build it with make. It should compile fine. Finally, there is the executable smrender and the libsmfilter.so. The latter is not mandatory for running *Smrender* since it may just be loaded dynamically by the rule set (see Section 8.1). Those files may be installed into the appropriate directories on your system with sudo make install.[28]
Smrender is known to compile with gcc 4.x on Debian Linux (Lenny, Squeeze, and Wheezy), FreeBSD version 8.x - 10.x, OpenBSD 5.x, and Mac OSX. It should compile on most Unixoid plattforms without further troubles, maybe even on Windows with Cygwin.

B. Writing Own Rendering Functions

The *Smrender* package includes a skeleton library in the directory libskel/. It implements the library constructor and destructor, and the actual rule function together with its initialization and de-initialization functions.
The directory contains also a Makefile which shows how to compile the library.

Smrender exports several functions which may be called by the library. The following list shows the most imported ones. The prototypes are defined in smrender.h, smlog.h, or osm_inplace.h.

```
// Use smrender's standard logging. This function is defined in 'smlog.h' and
// works similar to syslog(3).
void log_msg(int, const char*, ...);

// Get an OSM object (OSM_NODE, OSM_WAY, OSM_REL) with the specifed id. This
// function returns a pointer to either an osm_node_t or osm_way_t or_osm_rel_t
// structure on success, or NULL on error.
void *get_object(int, int64_t);
```

[26] http://www.cairographics.org/
[27] http://www.abenteuerland.at/download/smrender/
[28] Root privileges are required to install.

```
// Add an OSM object to the memory. The function returns 0 on success,
// otherwise -1 is returned. Preexisting objects with the same id are simply
// overwritten.
int put_object(osm_obj_t*);

// These functions return unique ids for nodes and ways.
int64_t unique_node_id(void);
int64_t unique_way_id(void);

// Initialize an OSM object. The number of tags (type short) and the number of
// node references (type int) must be supplied. Currently, the functions always
// return a valid pointer to an object. The objects returned are just partially
// initialized (see 'osm_func.c').
osm_node_t *malloc_node(short);
osm_way_t *malloc_way(short, int);
osm_rel_t *malloc_rel(short, short);
```

C. FAQ

This section covers some questions and answer which might arise.

C.1. Why is Smrender not written in C++?

On closer examination, the software architecture suggests an object-oriented programming language such as C++ but *Smrender* is written in C. The short answer is that C is always my first choice and the code was already too mature to switch to C++ without a high effort. The long answer is that *Smrender* is able to dynamically link libraries at runtime. Interfacing from C++ to a library written in C (currently) seems to be difficult (although not impossible).
But I still have in mind to rewrite *Smrender* in C++ when time comes.

C.2. Is it possible to create other maps, such as road maps?

Yes of course! The appearance of the map solely depends on the ruleset. A very simple first "land" ruleset is found in the rules directory. The only thing which is fixed is that *Smrender* uses a Mercartor projection which typically is not used for "land maps".

D. ToDo

Software is never finished and I have a long list of ideas in mind which I could implement.

- Rendering of rotated (not North-up) maps.
- Dynamic rules; these are rules which are generated during the rendering itself and are applied subsequently.
- Automatic font size for bays and capes (similar to the font size selection algorithm of polygons/islands).
- *Smrender* as an OSM database server with a Websockets interface (development started with version 4.0).